

The Backwards and Differencing Operators

It is convenient to write equations using the *backwards operator* B .

This is defined simply to be such that $BX_t = X_{t-1}$.

Then, we can write things such as $B^k X_t = X_{t-k}$.

The backwards operator applied to a constant is just equal to that constant.

We can rewrite the MA(q) model as

$$X_t - \mu = (1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q) \epsilon_t$$

$$\text{MA}(1): X_t - \mu = (1 + \theta B) \epsilon_t = \epsilon_t + \theta B \epsilon_t = \epsilon_t + \theta \epsilon_{t-1}$$

We can rewrite the AR(p) model as

$$(1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p) (X_t - \mu) = \epsilon_t$$

$$\text{AR}(1): (1 - \phi B)(X_t - \mu) = \epsilon_t$$

$$X_t - \mu - \phi B X_t + \phi B \mu = \epsilon_t$$

$$X_t - \mu - \phi X_{t-1} + \phi \mu = \epsilon_t$$

$$X_t = \mu + \phi (X_{t-1} - \mu) + \epsilon_t$$

1 We also define the *differencing operator* $\Delta = 1 - B$:

2
$$\Delta X_t = X_t - BX_t = X_t - X_{t-1}$$

3 Then,

4
$$\Delta^2 X_t = \Delta(X_t - X_{t-1}) = X_t - 2X_{t-1} + X_{t-2}$$

5 Note that applying the differencing operator twice is **not** equivalent
6 to taking the *lag two differences*: $X_t - X_{t-2}$

7 In R, the function `diff()` applies differencing to time series data.

8 After specifying the series, then one has to give both the lag of the
9 differences **and** the number of times the differencing is to be applied.

10 For example, to apply Δ twice, use

11

```
> diff(seriesdata, 1, 2)
```

12 To find the lag two differences, use

13

```
> diff(seriesdata, 2, 1)
```

The ARMA(p, q) Models

It is natural to think to combine the AR(p) and MA(q) models into a single process, the ARMA(p, q) model.

This can be written as

$$X_t = \mu + \sum_{i=1}^p \phi_i (X_{t-i} - \mu) + \underbrace{\epsilon_t + \sum_{j=1}^q \theta_j \epsilon_{t-j}}_{\text{MA}(q)}$$

where $\{\epsilon_t\}$ is WN($0, \sigma_\epsilon^2$).

Using the backwards operator, this can be written as

$$(1 - \phi_1 B - \dots - \phi_p B^p) \underbrace{(X_t - \mu)}_{\text{AR}(p)} = (1 + \theta_1 B + \dots + \theta_q B^q) \underbrace{\epsilon_t}_{\text{MA}(q)}$$

The conditions placed on ϕ in order to make the process stationary are the same as before.

1 **Fitting the ARMA(p, q) Process**

2 The ARMA(p, q) model is fit much like the previous models: Use the

3 function `arima()` with `order = c(p, 0, q)`.

4 The function `ARMAacf()` can still be used, with the arguments `ar`

5 and `ma` specifying the list of ϕ and θ parameters, respectively.

6 The values of p and q will typically be chosen via AIC.

1 The ARMA(1,1) Process

2 The ARMA(1,1) model achieves a good balance between simplicity
3 and quality of fit to real data; it does get used in practice.

4 The model can be written as

$$5 \quad X_t = \mu + \phi X_{t-1} + \theta \epsilon_{t-1} + \epsilon_t$$

6 and it can be shown that (see Section 12.8.3 in Ruppert):

$$7 \quad \gamma_X(0) = \frac{(1 + \theta^2 + 2\phi\theta)\sigma_\epsilon^2}{1 - \phi^2}$$

$$8 \quad \rho_X(1) = \frac{(1 + \phi\theta)(\phi + \theta)}{1 + \theta^2 + 2\phi\theta}$$

$$9 \quad \rho_X(k) = \phi \rho_X(k-1) \text{ for } k \geq 2$$

10 Example 12.9 in Ruppert shows a case where AIC concludes that the
11 best choice for a particular time series is an ARMA(1,1) model.

The ARIMA(p, d, q) Models

The final step in this progression of models is to incorporate *differencing* into the fit.

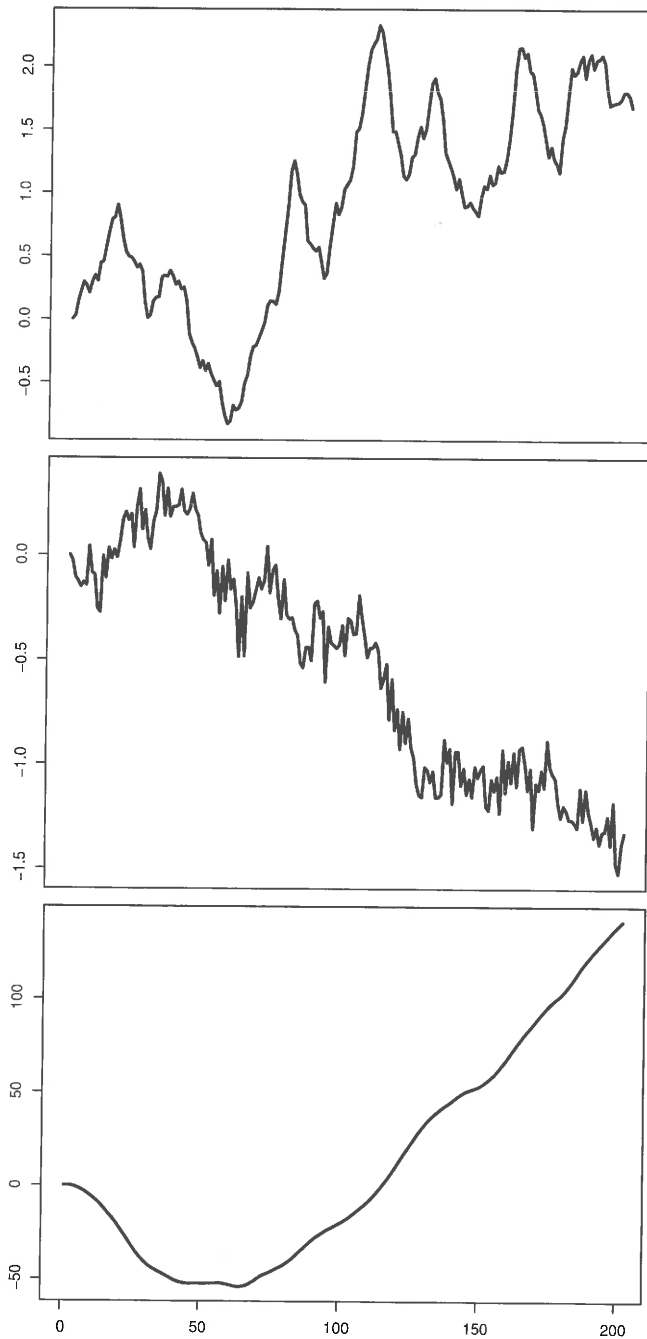
We saw back in Part 1 that time series which are clearly non-stationary can be transformed into (approximately) stationary processes via differencing. These are reproduced on the following pages.

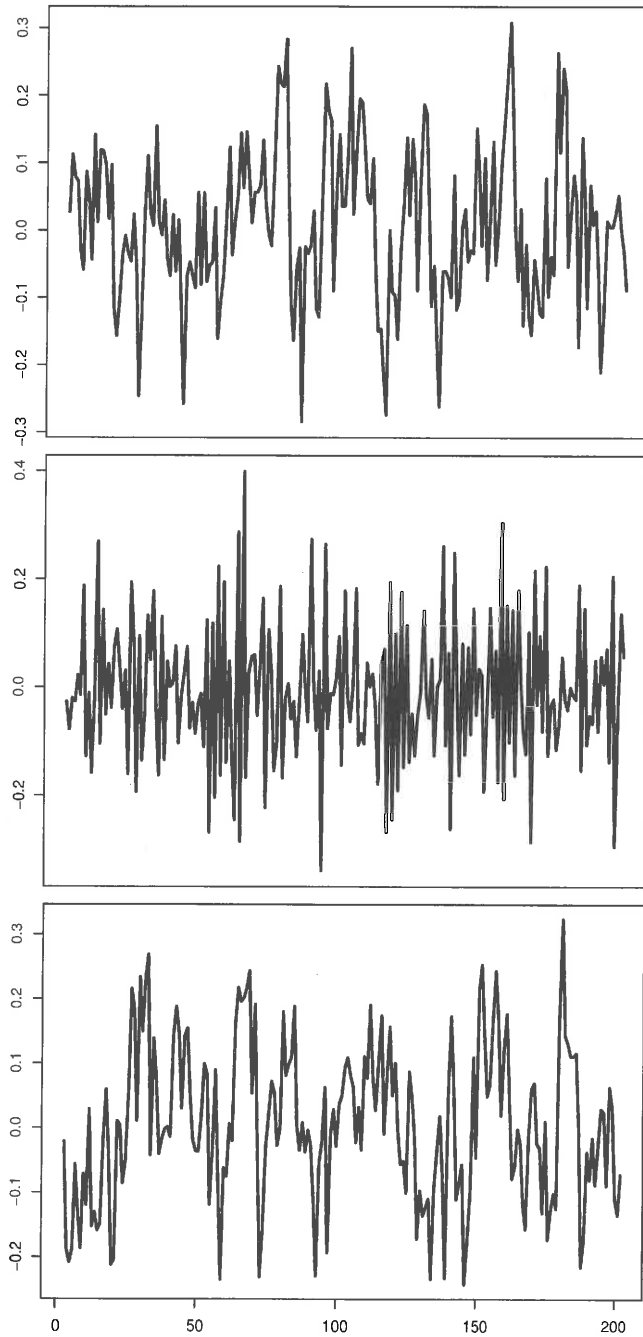
In some cases, differencing the series once is sufficient (the first two examples on the following pages), while in other cases the differencing may be applied two or more times. (The third example on the following page applies differencing twice.)

The argument d specifies the number of times differencing is applied.

Simply stated, a process $\{X_t\}$ is ARIMA(p, d, q) if $\Delta^d X_t$ is ARMA(p, q).

Of course, this implies that the ARIMA($p, 0, q$) model is equivalent to the ARMA(p, q) model.





1 **Exercise:** What does the "I" stand for in ARIMA?

2 "I" = integrated

3 taking differences is like taking derivatives

11 **Exercise:** Suppose that a time series has a trend that is well-fit by a
12 p^{th} degree polynomial, but if that trend is fit with a $(p - 1)^{st}$ degree
13 polynomial, a significant trend is still evident. How many differences
14 should be taken in order to get a process with a constant mean? How
15 many differences should be taken in order to get a process with a
16 mean of zero?

17 p^{th} order polynomial needs p derivatives (p diffs)
18 to be constant ^{mean} $\approx$ $p+1$ diffs to have
19 mean 0

1 Fitting an ARIMA Model

2 We will again use the function `arima()` to fit these models, but with
3 `order=c(p, d, q)`.

4 The first step in the process is to choose d .

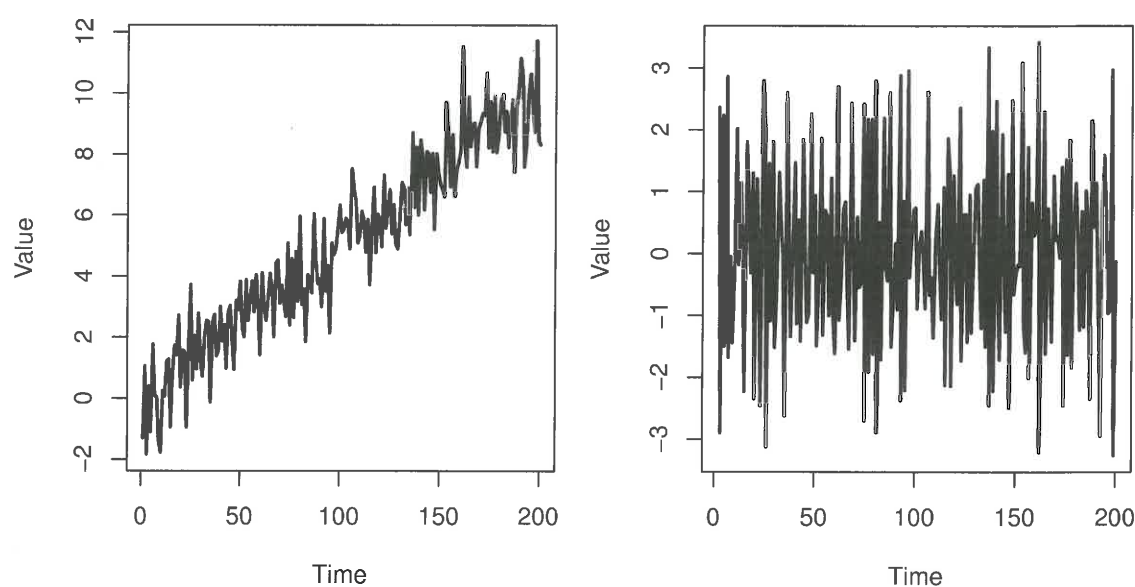
5 Since the choice of d is motivated by trying to transform into a sta-
6 tionary process, AIC is not a useful tool. Instead, one should use a
7 combination of visual inspection of the differenced time series, and
8 formal hypothesis tests (such as the ADF test) to determine if the
9 differenced series is approximately stationary.

10 **Important Point:** The function `arima()` assumes that, if differenc-
11 ing is applied ($d \geq 1$), then the differenced process has a **mean of**
12 **zero**. This is called the *zero drift assumption*.

13 Note, however, that if no differencing is applied ($d = 0$), then `arima()`
14 does fit a *non-zero drift* to the process.

Hence, if `arima()` is to be used, the zero-drift assumption may force an extra differencing step. In fact, however, in many cases this is unnecessary.

Figure shows a case where the original series (on the left) has a clear linear trend. Once the first difference is taken, the resulting series seems to have a mean close to zero.



Exercise: What happened in this case?

The error (the fluctuations around the trend)
 swamps the increase
 $0 < E(X_t - X_{t-1}) \ll SD(X_t - X_{t-1})$

Comment: Do not apply more differencing than is required. In addition to creating unwarranted additional complexity, an extra differencing step induces a significant negative correlation at lag one.

The example in Figure 4 below illustrates the effect. The original series is IID white noise. The ACF of the original series and the first-differenced series are shown.

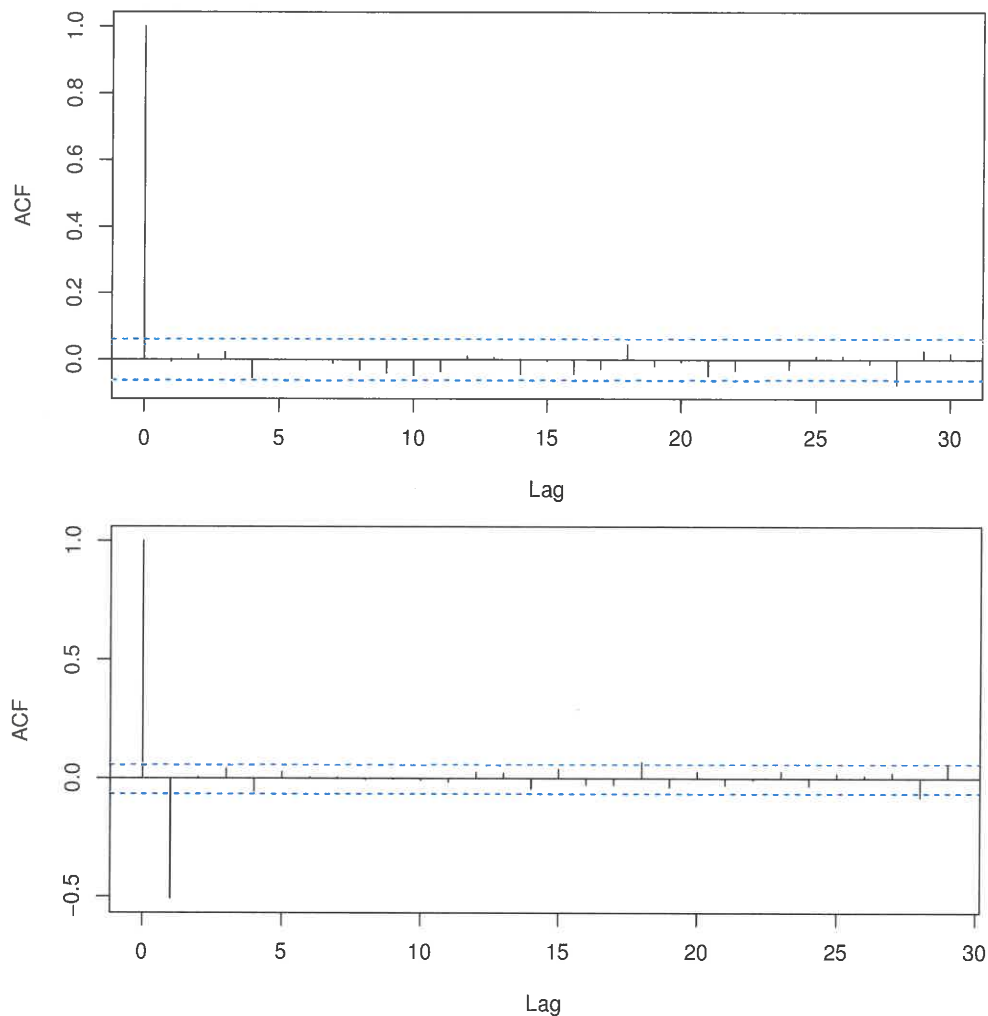


Figure 4: The ACF of the original and first-differenced series, in a case where the original series is IID white noise.

$$X_t \sim \text{iid } N(0, 1)$$

$$Y_t = X_t - X_{t-1}$$

$$\begin{aligned} \text{Cov}(Y_t, Y_{t-1}) &= \text{Cov}(X_t - X_{t-1}, X_{t-1} - X_{t-2}) \\ &= E((X_t - X_{t-1})(X_{t-1} - X_{t-2})) \\ &= -E(X_{t-1}^2) = -1 \end{aligned}$$

$$V(Y_t) = V(X_t - X_{t-1}) = 2V(X_t) = 2$$

$$\text{Cor} = -\frac{1}{2}$$

- 1 Once d is chosen, then p and q can be chosen via AIC.
- 2 The already-mentioned function `auto.arima()`, part of the pack-
3 age `forecast` can be used to automate the search for the optimal
4 p and q . This is a powerful function with a great deal of built-in
5 capabilities, but one should be fully aware of its assumptions and
6 approaches.
- 7 Here are some important points regarding its use. Most of these
8 could be found by looking at the `help()` function.
- 9 • The default is that the function uses a stepwise search, incremen-
10 tally changing the values of p and q to find the optimal model.
11 This can be changed to an exhaustive search by setting `stepwise`
12 `= FALSE`.
 - 13 • The default is to consider models with seasonal components. If
14 you want to remove that, set `seasonal = FALSE`.
 - 15 • The maximum value of p which is considered is 5; this can be
16 changed by setting `max.p`.
 - 17 • The maximum value of q which is considered is 5; this can be
18 changed by setting `max.q`.

- 1 • If an exhaustive search is used, the value of $p + q$ cannot exceed
2 5. This can be changed by setting `max.order`.
- 3 • Unlike `arima()`, the function `auto.arima()` **does** allow for
4 non-zero drift. If you want to change this, then set `allowdrift`
5 = `FALSE`. The decision to include/exclude μ is also made based
6 on AIC.
- 7 • If you set `trace = TRUE`, then you can see the different values
8 of AIC for all of the models fit.
- 9 • The optimization criterion is set using argument `ic`. The possible
10 choices are `aicc` (the default), `aic`, and `bic`.
- 11 • The default is to allow for the fit of models which are **not** station-
12 ary. If you want to force a stationary model, then set `stationary`
13 = `TRUE`.
- 14 • If leave `approx = TRUE` (the default), then the code does not
15 use exact likelihood calculations.

1 **Comment:** The function `auto.arima()` **does** also incorporate au-
2 tomatic selection of d . It makes this selection by performing a se-
3 quence of hypothesis tests for stationarity, starting with $d = 0$, then
4 with $d = 1$, and so forth.

5 In practice, however, this procedure can yield strange results. One
6 reason for this is that the tests are performed with the null hypothesis
7 being that the process is stationary, and α is fixed at 0.01 for this
8 test (it cannot be changed). The result is that some time series with
9 clear non-stationary behavior can fail to reject because of the high
10 standard for rejection.

11 The recommendation is to not rely upon this procedure for choosing
12 d , but instead make a more holistic determination.

13 The value of d can be fixed by using the argument `d` to `auto.arima()`.

- 1 **R Exercise:** Obtain the data from the St. Louis Fed on the "Trade
2 Weighted U.S. Dollar Index: Major Currencies (DTWEXM)" by using
3 the following commands:

```
4 > DTWEXM = getSymbols("DTWEXM", src="FRED", auto.assign=F)  
5 > DTWEXM = DTWEXM["2017"]
```

- 6 This is described as "A weighted average of the foreign exchange
7 value of the U.S. dollar against a subset of the broad index curren-
8 cies that circulate widely outside the country of issue. Major curren-
9 cies index includes the Euro Area, Canada, Japan, United Kingdom,
10 Switzerland, Australia, and Sweden. "

- 11 Fit an ARIMA model to these data.

decided to apply one difference operation, removed ^{NAs}
using defaults, `auto.arima()` chose $p=0, q=0, \mu=0$

with `approx=FALSE`, $\mu \neq 0$

with `approx=FALSE`, `stepwise=FALSE`, $\mu \neq 0$

Fail to find enough serial dependence to
fit an ARIMA model

1 **Part 4: Forecasting**

2 Text references: §12.12 in Ruppert and Matteson

3 The act of predicting future observations of a time series is called
4 *forecasting*. As already mentioned, this is a fundamental motivation
5 for the use of time series models.

6 In this Part we will present forecasting the context of ARIMA models,
7 but it is typically not difficult to extend these ideas to other models;
8 we'll see this in the future as we consider these models.

9 The basic idea of forecasting is simple: We have fit a model to the
10 observed series. These models predict the current value of the series
11 as a function of previous values and error terms. We will simply
12 apply these models to the most recent data to predict future values.

13 Construction of *prediction intervals* or *predictive distributions* is crucial:
14 There is typically a lot of noise in these processes, and we want to
15 accurately state our uncertainty in our predictions.

16 As always, we will rely on R to do the heavy computations.

Forecasting with AR(p) Models

Assume that our observed series runs up until time n .

For the case of an AR(1) model, we know that

$$X_{n+1} = \mu + \phi(X_n - \mu) + \epsilon_{n+1}$$

so the natural forecast for this value is

$$\hat{X}_{n+1} = \hat{\mu} + \hat{\phi}(X_n - \hat{\mu})$$

Note that since ϵ_{n+1} has a mean of zero, our best prediction of its value is zero. Of course, ϵ_{n+1} **does** play an important role in calculating the standard error in this prediction.

Exercise: Assume $k > 0$. Derive a general form for the forecast for X_{n+k} in the case of an AR(1) process $\{X_t\}$.

$$\begin{aligned} k=2: X_{n+2} &= \mu + \phi(X_{n+1} - \mu) + \epsilon_{n+2} \\ \hat{X}_{n+2} &= \hat{\mu} + \hat{\phi}(\hat{X}_{n+1} - \hat{\mu}) \\ &= \hat{\mu} + \hat{\phi}(\hat{\mu} + \hat{\phi}(X_n - \hat{\mu}) - \hat{\mu}) \\ &= \hat{\mu} + \hat{\phi}^2(X_n - \hat{\mu}) \end{aligned}$$

$$\text{In general: } \hat{X}_{n+k} = \hat{\mu} + \hat{\phi}^k(X_n - \hat{\mu})$$

- 1 **Exercise:** Assume that the AR(1) process is stationary. What happens
2 to $\hat{X}_{n+k}$ as k increases? Does this seem reasonable?

3 stationary $\Rightarrow |\hat{\phi}| < 1$, so $\hat{\phi}^k \rightarrow 0$ as $k \rightarrow \infty$

4 so, $\hat{X}_{n+k} \rightarrow \hat{\mu}$ as k increases

6 For "large" k , our best forecast of
7 X_{n+k} is simply $\hat{\mu}$

9 Reasonable since $E(X_{n+k}) = \mu$

- 14 The idea extends simply to the AR(p) process. In general,

15
$$\hat{X}_{n+1} = \hat{\mu} + \hat{\phi}_1(X_n - \hat{\mu}) + \hat{\phi}_2(X_{n-1} - \hat{\mu}) + \cdots + \hat{\phi}_p(X_{n-p} - \hat{\mu})$$

Forecasting with ARMA(p, q) Models

Again, start with a simple model, the MA(1) model. In this case we know that

$$X_{n+1} = \mu + \epsilon_{n+1} + \theta\epsilon_n$$

and our forecast is

$$\widehat{X}_{n+1} = \widehat{\mu} + \widehat{\theta}\widehat{\epsilon}_n$$

Note that here we've replaced ϵ_n with the n^{th} residual.

In the MA(1) case, as soon as we move beyond the first step beyond n , our forecast is simply $\widehat{\mu}$. The reason for this is that

$$X_{n+k} = \mu + \epsilon_{n+k} + \theta\epsilon_{n+k-1}$$

and we do not have estimates of either ϵ_{n+k} or ϵ_{n+k-1} when $k \geq 2$.

1 **Exercise:** Derive $\hat{X}_{n+k}$ for $k \geq 1$ when $\{X_t\}$ is ARMA(1, 1).

2
$$X_t = \mu + \phi(X_{t-1} - \mu) + \theta \epsilon_{t-1} + \epsilon_t$$

3
$$X_{n+k} = \mu + \phi(X_{n+k-1} - \mu) + \theta \epsilon_{n+k-1} + \epsilon_{n+k}$$

5 when $k=1$,

6
$$\hat{X}_{n+1} = \hat{\mu} + \hat{\phi}(X_n - \hat{\mu}) + \hat{\theta} \hat{\epsilon}_n$$

9 when $k > 1$,

10
$$\hat{X}_{n+k} = \hat{\mu} + \hat{\phi}(\hat{X}_{n+k-1} - \hat{\mu})$$

Forecasting with ARIMA(p, d, q) Models

Extending from the ARMA(p, q) case to the ARIMA(p, d, q) case is not difficult. Simply recall that if $\{X_t\}$ is ARIMA(p, d, q), then $Y_t = \Delta^d X_t$ is ARMA(p, q).

Hence, one can start by making the forecasts for Y_{n+k} for $k \geq 1$. Then, “integrate” to get back the forecasts for X_{n+k} .

For instance, if $d = 1$, then

$$Y_{n+1} = \Delta X_{n+1} = X_{n+1} - X_n$$

so the forecast for Y_{n+1} could be turned into a forecast for X_{n+1} via

$$\hat{X}_{n+1} = \hat{Y}_{n+1} + X_n$$

Exercise: Show that

$$\hat{X}_{n+2} = \hat{Y}_{n+2} + \hat{Y}_{n+1} + X_n$$

The forecast for $\hat{X}_{n+2}$ is found by adding the next two “steps” $\hat{Y}_{n+1}$ and $\hat{Y}_{n+2}$ onto X_n

1

2 **Standard Errors of Forecasts**

3 As was the case with regression, we consider predictions (forecasts)
4 without errors attached to them to be practically useless.

5 In the particular case of time series, we note that the machinery pre-
6 sented in the previous pages would allow one to forecast the value
7 of the series an arbitrary length of time into the future. Yet, it should
8 be clear that the quality of these forecasts could deteriorate quickly
9 when we move a few time steps into the future.

10 This does not bring into question the value of time series; it is often
11 the case that these forecasts are done “in real time,” and that one is
12 really only ever making a one-step forecast.

13 We will rely on software to do the calculations, but it is instructive to
14 consider a simple case.

Standard Errors for AR(1) Forecasts

Assume that we are working with a stationary AR(1) process $\{X_t\}$.

It is useful to consider the following decomposition:

$$\begin{aligned} X_{n+1} - \widehat{X}_{n+1} &= (\mu + \phi(X_n - \mu) + \epsilon_{n+1}) - (\widehat{\mu} + \widehat{\phi}(X_n - \widehat{\mu})) \\ &= (\mu - \widehat{\mu}) + (\phi - \widehat{\phi})X_n - (\phi\mu - \widehat{\phi}\widehat{\mu}) + \epsilon_{n+1} \end{aligned}$$

Now I can find

$$E((X_{n+1} - \widehat{X}_{n+1})^2) = V(X_{n+1} - \widehat{X}_{n+1})$$

Exercise: What quantities should be considered to be random in the derivation of

$$E((X_{n+1} - \widehat{X}_{n+1})^2)?$$

① Errors in estimation of μ and ϕ
 ($\widehat{\mu}, \widehat{\phi}$ are random variables)

② ϵ_{n+1} is random

When n is large, we can make a further approximation by assuming that the estimates of ϕ and μ will be close to their true values. Then,

$$\begin{aligned} X_{n+1} - \widehat{X}_{n+1} &= (\mu - \widehat{\mu}) + (\phi - \widehat{\phi}) X_n - (\phi\mu - \widehat{\phi}\widehat{\mu}) + \epsilon_{n+1} \\ &\approx \epsilon_{n+1} \end{aligned}$$

and hence the errors are dominated by the contribution from ϵ_{n+1} .

Taking this further, we see that

$$\begin{aligned} X_{n+k} - \widehat{X}_{n+k} &\approx (\mu + \phi(X_{n+k-1} - \mu) + \epsilon_{n+k}) - (\mu + \phi(\widehat{X}_{n+k-1} - \mu)) \\ &= \phi^{k-1}\epsilon_{n+1} + \phi^{k-2}\epsilon_{n+2} + \cdots + \phi\epsilon_{n+k-1} + \epsilon_{n+k} \end{aligned}$$

and hence

$$V(X_{n+k} - \widehat{X}_{n+k}) = \underbrace{\left(\frac{1 - \phi^{2k}}{1 - \phi^2} \right)}_{\sum_{i=0}^{k-1} \phi^{2i}} \sigma_\epsilon^2$$

and we note that as $k \rightarrow \infty$,

$$V(X_{n+k} - \widehat{X}_{n+k}) \rightarrow \frac{\sigma_\epsilon^2}{1 - \phi^2}. \quad (*)$$

Exercise: What does $(*)$ say about forecasts of values at long lags?

$$\text{For AR}(1), V(X_t) = \frac{\sigma_\epsilon^2}{1 - \phi^2}$$

For k large, our best forecast of X_{n+k} is $\widehat{\mu}$ and the variance is $\frac{\widehat{\sigma}_\epsilon^2}{1 - \widehat{\phi}^2}$

1

2 **Forecasting in R**

3 The `predict()` function will do the hard work of calculating fore-
4 casts and their standard errors.

5 The syntax is simple: If `holdfit` is the output from either `arima()`
6 or `auto.arima()` then

7

```
> holdfc = predict(holdfit, n.ahead=k)
```

8 will predict the next k values of the time series.

9 In particular, `holdfc` consists of two components: `holdfc$pred`
10 which holds the forecasts, and `holdfc$se` which holds the standard
11 errors for each of the forecasts.

1 Example: AAA Corporate Bond Yield

2 For this example, I will use “Moody’s Seasoned Aaa Corporate Bond
3 Yield” data, obtained from the St. Louis Fed. The data are monthly,
4 and I will use from the period 1990 to 2012:

```
5 > AAA = getSymbols("AAA", auto.assign=F, src="FRED")  
6 > AAA = AAA["1990::2012"]
```

7 Figure 1 shows the original time series.

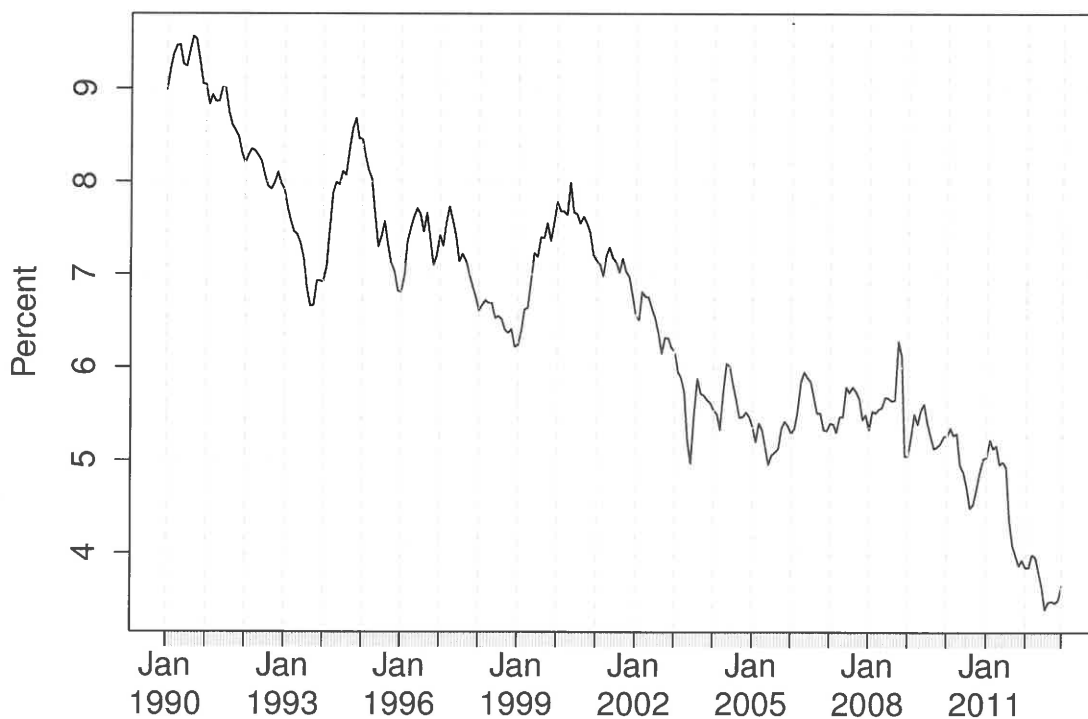


Figure 1: Seasoned Aaa Corporate Bond Yield for 1990 to 2012.

1 Clearly, something is needed to remove the trend. We will consider
2 using the first-differenced series, and inspect the plot of the series,
3 along with the result of the ADF test:

```
4 > AAAdiff1 = diff(AAA,1,1)
5 > adf.test(as.numeric(AAAdiff1)[-1], alt="s")
```

6

7

8 Augmented Dickey-Fuller Test

9

```
10 data:  as.numeric(AAAdiff1)[-1]
11 Dickey-Fuller = -6.4686, Lag order = 6,
12    p-value = 0.01
13 alternative hypothesis: stationary
```

14

15 Warning message:

```
16 In adf.test(as.numeric(AAAdiff1)[-1], alt = "s") :
```

```
17    p-value smaller than printed p-value
```

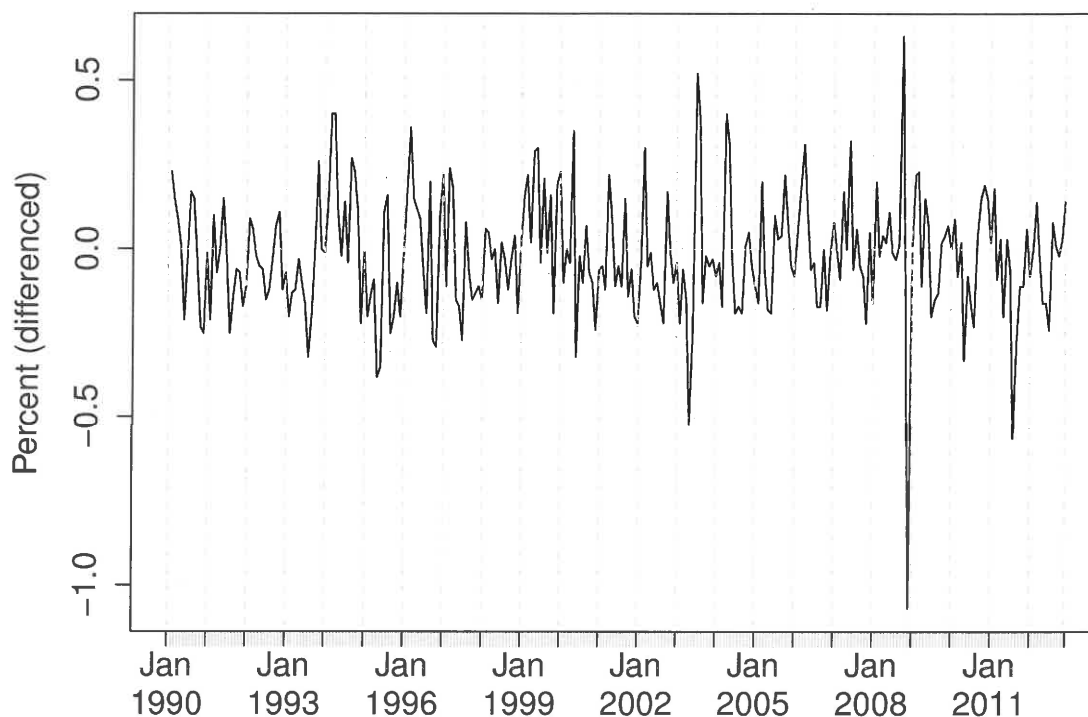


Figure 2: Seasoned Aaa Corporate Bond Yield for 1990 to 2012, first differenced.

Exercise: Does the first differencing seem to be sufficient?

Based on both the plot of the series
and the ADF test, first differencing
seems sufficient

1 Now, let's use `auto.arima()` to fit the model:

2 `> holdfit = auto.arima(AAA, d=1, approx=F)`

3 The result was as follows:

4 Series: AAA

5 ARIMA(0,1,1)

6

7 Coefficients:

8 ma1

9 0.2888

10 s.e. 0.0581

11

12 σ^2 estimated as 0.03014: log likelihood=91.27

13 AIC=-178.53 AICc=-178.49 BIC=-171.3

14 I checked the residual plots, and tested for serial correlation in the
15 residuals. There was no evidence of a problem with the fit of the
16 model.

1 Finally, let's forecast the next year of data for the series:

```
2 > holdpreds = predict(holdfit, n.ahead=6)
```

3 The result was as follows:

```
4 $pred
```

```
5 Time Series:
```

```
6 Start = 277
```

```
7 End = 282
```

```
8 Frequency = 1
```

```
9 [1] 3.690547 3.690547 3.690547 3.690547 3.690547 3.690547
```

```
10
```

```
11 $se
```

```
12 Time Series:
```

```
13 Start = 277
```

```
14 End = 282
```

```
15 Frequency = 1
```

```
16 [1] 0.1736047 0.2831947 0.3609153 0.4246416 0.4799804 0.5295675
```


- 1 **Exercise:** Note that the final value in this series is $X_n = 3.65$, and
2 the final residual is $\hat{\epsilon}_n = 0.140$. Confirm that the forecasts from R are
3 correct.

4 $\{\psi_t\}$ is the differenced series

$$\begin{aligned}\hat{\psi}_{n+1} &= \hat{\mu} + \hat{\theta} \hat{\epsilon}_n \\ &= 0 + (0.2888)(0.140) \\ &= 0.040\end{aligned}$$

$$\begin{aligned}\hat{X}_{n+1} &= X_n + \hat{\psi}_{n+1} \\ &= 3.65 + 0.040 \\ &= 3.69\end{aligned}$$

13 For $k > 1$, $\hat{\psi}_{n+k} = 0$ since $\hat{\mu} = 0$

15 So, $\hat{X}_{n+k} = \hat{X}_{n+1}$, for $k \geq 1$

Part 5: Seasonal Models

Text references: §13.1 in Ruppert and Matteson

Figure 1 shows twenty years of monthly housing starts (in thousands of units). There is clear trend (and hence nonstationarity), but there is also a strong *seasonal component* to the series. Such structure should be handled by special time series models.

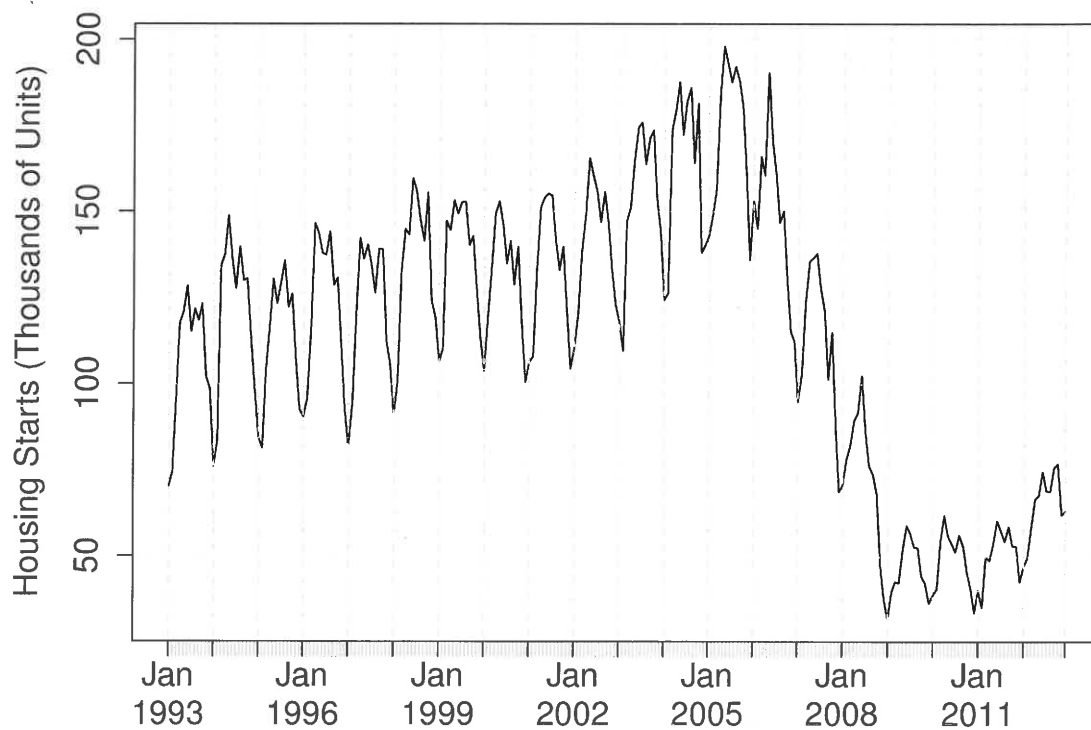


Figure 1: Monthly housing starts for Jan. 2004 to Dec. 2012.

- 1 The first differences of the housing starts is shown as Figure 2.

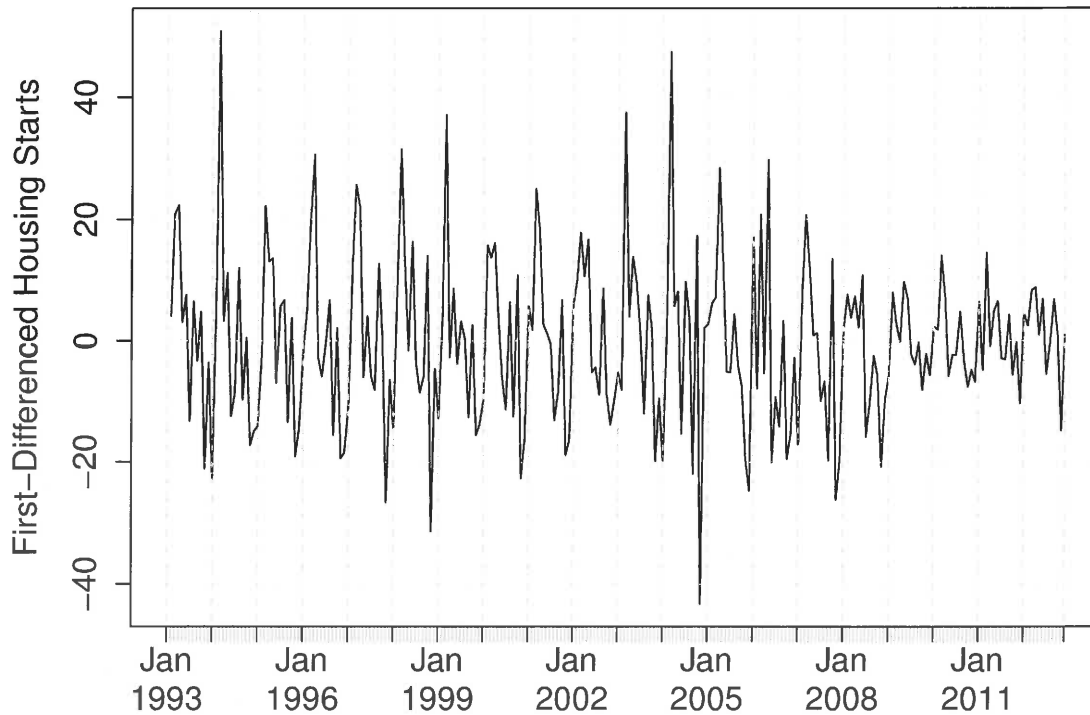


Figure 2: First differences of monthly housing starts series.

- 2 It is evident that although the trend has been largely removed, the
3 seasonal component remains.

Figure 3 shows the ACF of the first-differenced series. This plot shows clear evidence of the seasonal component: At lags of around six months, the correlation is negative, while at lags of around twelve months, the correlation is strongly positive. This pattern then repeats in yearly cycles.

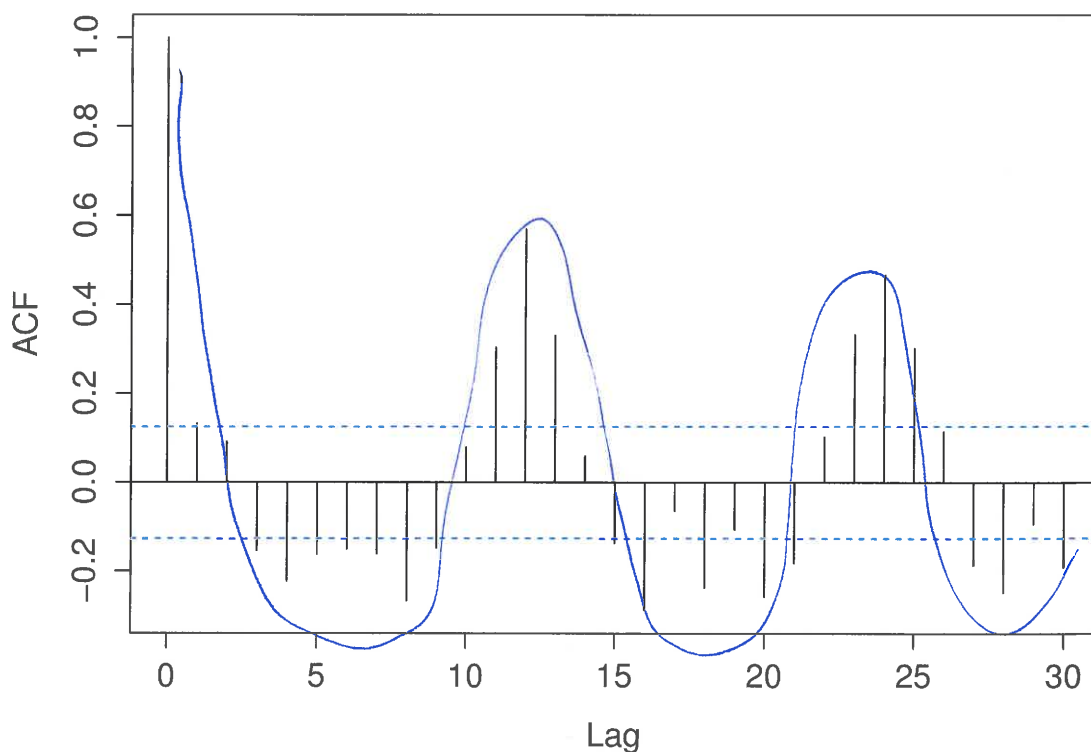


Figure 3: ACF of first differences of monthly housing starts series.

1 For reference, these are the R commands used to create the plots on
2 the previous pages:

```
3 library(quantmod)
4
5 HOUSTNSA = getSymbols("HOUSTNSA", src="FRED", auto.assign=F)
6 HOUSTNSA = HOUSTNSA['1993::2012']
7
8 # The raw series
9
10 postscript(file="HOUSraw.eps",width=8,height=6,hORIZ=F)
11 plot(HOUSTNSA,ylab="Housing Starts (Thousands of Units)",
12      main="",cex.axis=1.3,cex.lab=1.3)
13 dev.off()
14
15 # The first-differenced series
16
17 HOUSdiff = diff(HOUSTNSA,1,1)
18 postscript(file="HOUSdiff.eps",width=8,height=6,hORIZ=F)
19 plot(HOUSdiff,ylab="First-Differenced Housing Starts",
20      main="",cex.axis=1.3,cex.lab=1.3)
21 dev.off()
22
23 # ACF of first differences
24
25 postscript(file="HOUSdiffacf.eps",width=8,height=6,hORIZ=F)
26 acf(HOUSdiff,na.action=na.pass,main="",cex.axis=1.3,
27     cex.lab=1.3,lag.max=30)
28 dev.off()
```

- 1 A natural idea at this point is to consider taking the lag 12 differences
2 of the series. See Figure 4.

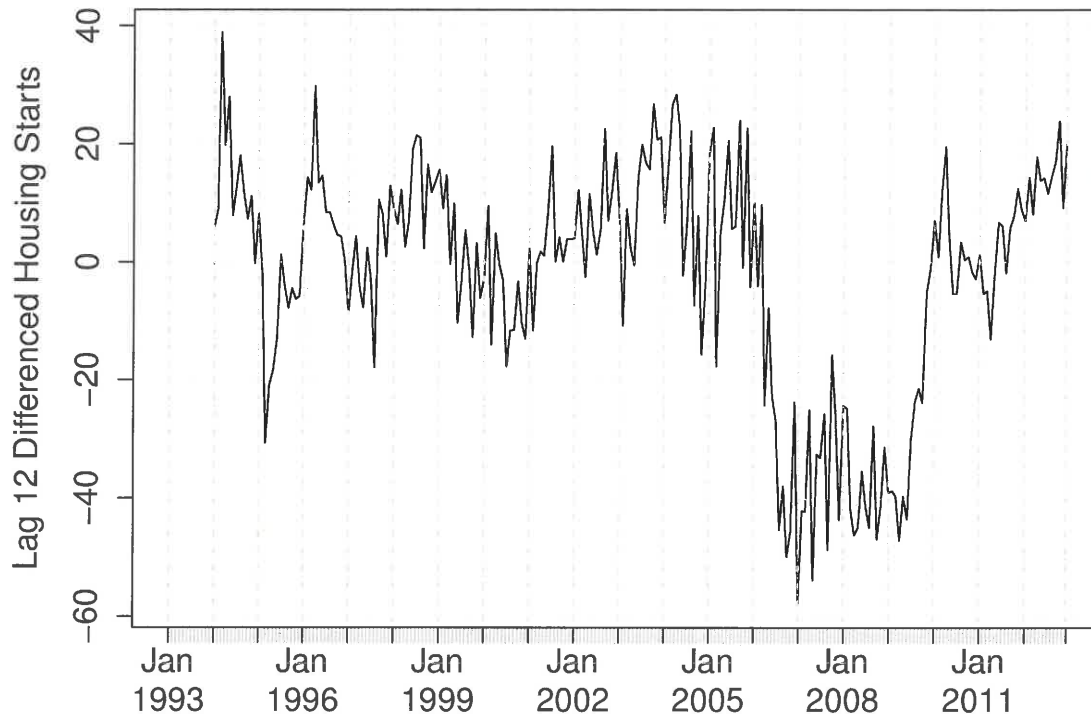


Figure 4: Lag 12 differences of monthly housing starts series.

- 3 Note that the command in R to take lag 12 differences is

4 `> diff(HOUSTNSA, 12, 1)`

- 5 The use of such differences is important in seasonal models. We will
6 extend our previous notation, and define

7 $\Delta_s X_t = (1 - B^s)X_t = X_t - X_{t-s}$

1 The trend is still evident in Figure 4. So, one could consider tak-
 2 ing the first differences of the lag 12 differenced series; the result is
 3 shown in Figure 5.

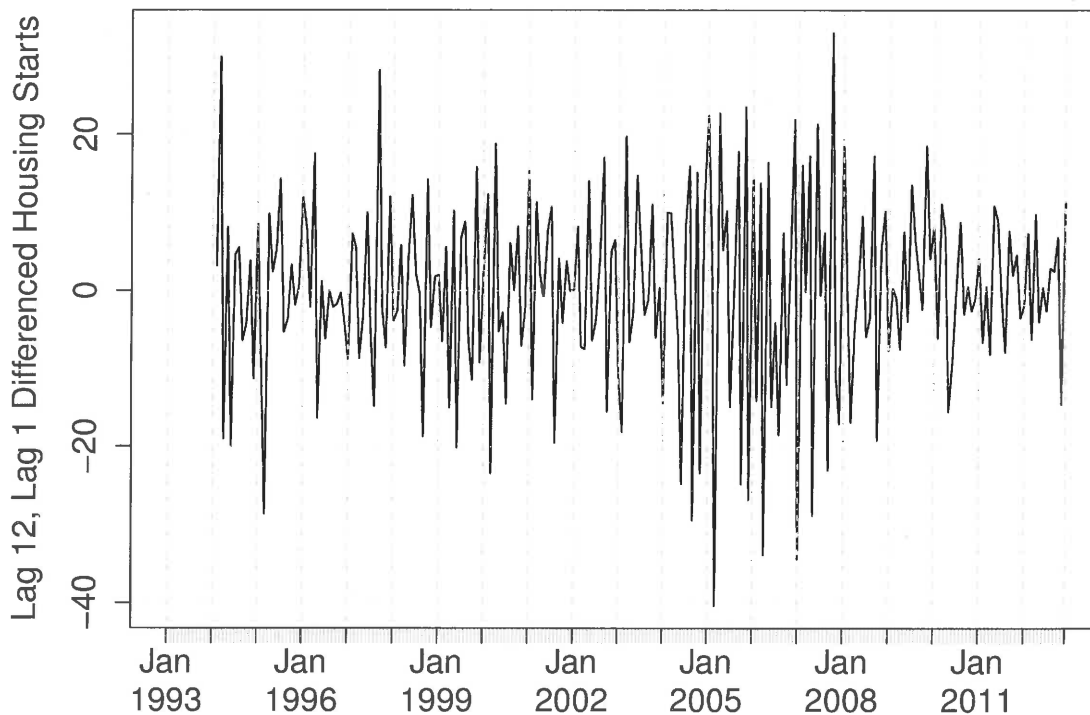


Figure 5: Lag 12, Lag 1 differences of monthly housing starts series.

4 **Exercise:** Does it matter in which order the differencing was applied?

5 No. $\Delta_{12} \Delta = \Delta \Delta_{12} \neq \Delta^{13} \neq \Delta_{13}$

6

7

8

9

The Multiplicative ARIMA Model

One could consider fitting an ARMA model to the series shown in Figure 5, but in practice this often turns out to not yield models with sufficient flexibility.

In particular, we note that in this series there are two major serial dependencies present: That arising from the seasonal component, and that from the usual, consecutive time-step dependence. One might guess that these two components should require different “models” in order to capture the structure sufficiently well.

This is what is accomplished by the *multiplicative ARIMA models*.

For this model, one will specify both the order for the “nonseasonal” and the “seasonal” components. The usual notation is something such as

$$\text{ARIMA}((p, d, q) \times (P, D, Q)_s)$$

Here, (p, d, q) is the order for the nonseasonal component, (P, D, Q) is the order for the seasonal component, and s is the period of the series. In our example, $s = 12$ (months).

To demonstrate the construction of the model, start with a simple example, the $\text{ARIMA}((1, 1, 0) \times (1, 1, 0)_s)$ process:

$$\underbrace{(1 - \phi B)}_{\substack{\text{non-seasonal} \\ \text{AR}(1)}} \underbrace{(1 - \phi^* B^s)}_{\substack{\text{seasonal} \\ \text{AR}(1)}} (\Delta_s(\Delta X_t) - \mu) = \epsilon_t$$

Exercise: Decompose the above model to justify its form.

$$\begin{aligned} (1 - \phi B)(1 - \phi^* B^s)(\Delta_s(\Delta X_t) - \mu) &= \epsilon_t \\ (1 - \phi B - \phi^* B^s + \phi \phi^* B B^s)(\Delta_s(\Delta X_t) - \mu) &= \epsilon_t \end{aligned}$$

ϕ is the coef. on $\Delta_s \Delta X_{t-1}$

ϕ^* is the coef on $\Delta_s \Delta X_{t-s}$

$\phi \phi^*$ is the coef. on $\Delta_s \Delta X_{t-s-1}$

In the preceding example, $p=1$, $P=1$

What if $P=2$, $p=1$. What time steps are included in the model?

Y_t is modelled as a function of

$$Y_{t-1}, Y_{t-s}, Y_{t-s-1}, Y_{t-2s}, Y_{t-2s-1}$$


Since $p=1$

- 1 The general form for the ARIMA((p, d, q) × (P, D, Q)_s) is written as
- 2 follows:

$$\begin{aligned}
 & \left(1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p\right) \left(1 - \phi_1^* B^s - \dots - \phi_p^* (B^s)^p\right) \\
 & \quad \times \left(\Delta_1^d \Delta_s^D X_t - \mu\right) \\
 & = \left(1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q\right) \left(1 + \theta_1^* B^s + \dots + \theta_q^* (B^s)^q\right) \\
 & \quad \times \epsilon_t
 \end{aligned}$$

The "multiplicative ARIMA model"

Fitting the Multiplicative ARIMA Model in R

We will consider two options for fitting the multiplicative ARIMA model in R: With `arima()`, and with `auto.arima()`.

With function `arima()`, one simply includes an additional argument `seasonal` which specifies P , D , and Q , along with s :

```
seasonal = list(order=c(P,D,Q), period=s)
```

For example, in our housing starts example, if one wanted to fit an ARIMA $((1,1,1) \times (1,1,0)_{12})$ model, the command would be as follows:

```
> holdfit = arima(HOUSTNSA, order=c(1,1,1),  
seasonal=list(order=c(1,1,0), period=12))
```

1 The output of the model appears as follows:

2 Series: HOUSTNSA

3 ARIMA(1,1,1) (1,1,0) [12]

4

5 Coefficients:

6 ar1 ma1 sar1

7 -0.1537 -0.4479 -0.3989

8 s.e. 0.1004 0.0834 0.0609

9

10 sigma^2 estimated as 90.63: log likelihood=-834.85

11 AIC=1677.7 AICc=1677.88 BIC=1691.4

12 Of course, one should inspect the residuals from this fit to assess the
13 quality of the fit.

- 1 The use of the `predict()` function for forecasting is identical to that
 2 shown previously.

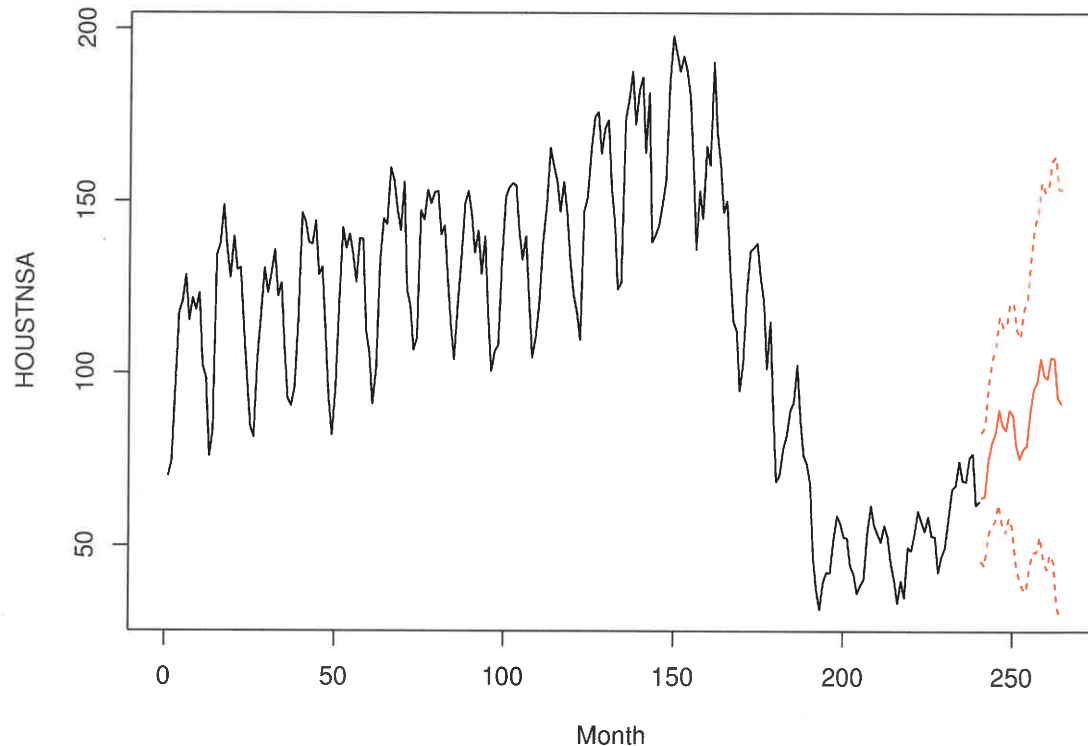


Figure 6: Series with the forecasts for the following 24 months.

- 3 The R commands used to create this plot were as follows:

```

4 holdpreds = predict(holdfit, n.ahead=24)
5
6 HOUSTs = ts(HOUSTNSA)
7
8 plot(HOUSTs, xlab="Month", xlim=c(0,264))
9 lines(holdpreds$pred,col=2)
10 lines(holdpreds$pred+(1.96*holdpreds$se),col=2,lty=2)
11 lines(holdpreds$pred-(1.96*holdpreds$se),col=2,lty=2)

```

1 **The Use of `auto.arima()` with Seasonal Models**

2 The function `auto.arima()` allows for automatic selection of P and
3 Q (in addition to p and q) using AIC or related criteria.

4 Note that one does not automatically select s . Typically, s is “known”
5 or “obvious” given the context of the problem, or from inspection of
6 the time series or its ACF.

7 So, `auto.arima()` assumes that the time series that is passed in
8 has already been represented in a way that specifies the period of the
9 series.

10 To do this, convert the `xts` object into a standard `ts` object with the
11 frequency correctly specified. In our example:

```
12 > HOUSTs = ts(HOUSTNSA, frequency=12)
```

1 **Comment:** You will note that `arima()` and `auto.arima()` have
2 an unfortunate difference in that, for the same problem, in one we
3 set `period=12`, and in the other `frequency=12`.

4 Here is how to think about it:

5 • When we called `arima()`, the time unit for the series is month,
6 so the period is correctly set at 12: There are 12 months in one
7 cycle of the seasonal component.

8 • The function `auto.arima()` assumes that the provided series
9 is formatted such that the period of the seasonal component is
10 equal to a **single time unit**. So, in our case, in order to achieve
11 this, the period would have to be **one year**.

12 So, by setting `frequency=12`, we are stating that there are 12
13 observations in the series **per time unit**.

14 Confusingly, when using `arima()`, if the argument `period` is not
15 specified, the default is to use the `frequency()` of the provided
16 series.

1 As before, the choice of d and D should be based on inspection of the
2 resulting series, checking to see if the differenced series appears to be
3 approximately stationary. Excessive differencing is not a good idea.

4 In our example, if we stick with $d = 1$ and $D = 1$, then the command
5 we need is

```
6 > holdfit = auto.arima(HOUSTs, d=1, D=1, approx=F)
```

7 Note that I am disabling the “approximation” method so that true
8 maximum likelihood estimates are found.

```
9 Series: HOUSTs
```

```
10 ARIMA(0,1,3)(0,1,1)[12]
```

P Q P Q

```
11  
12 Coefficients:
```

```
13          ma1          ma2          ma3          sma1
```

```
14          -0.5194    0.0695    0.1078    -0.6953
```

```
15 s.e.      0.0687    0.0696    0.0653    0.0574
```

```
16
```

```
17 sigma^2 estimated as 73.93:  log likelihood=-814.63
```

```
18 AIC=1639.26    AICc=1639.54    BIC=1656.39
```

- 1 Below is the result of forecasting the next two years of data for the
- 2 series, using this improved model.

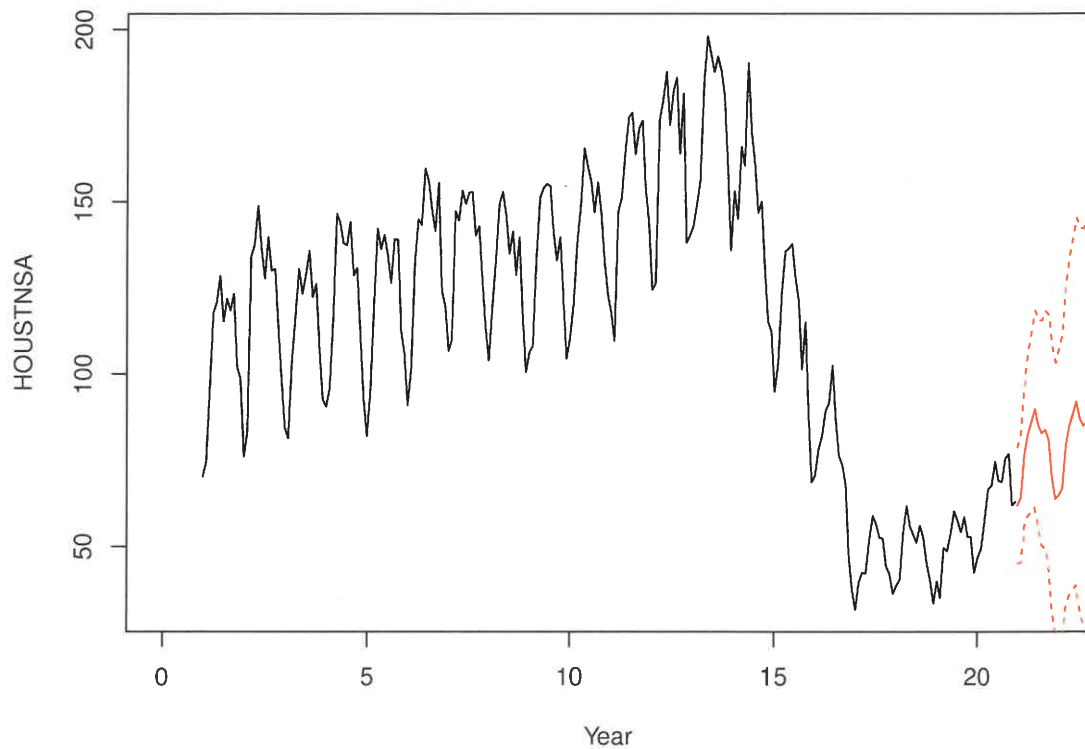


Figure 7: Series with the forecasts for the following 24 months.

```
3 holdpreds = predict(holdfit, n.ahead=24)
4
5 plot(HOUSTs, xlab="Year", xlim=c(0,22))
6 lines(holdpreds$pred,col=2)
7 lines(holdpreds$pred+(1.96*holdpreds$se),col=2,lty=2)
8 lines(holdpreds$pred-(1.96*holdpreds$se),col=2,lty=2)
```